(VERY) ROUGH DRAFT: A Unified Quantum Algorithm for Elementary Functions

Iain Burge
SAMOVAR, Institut Polytechnique de Paris, Palaiseau, France
iain-james.burge@telecom-sudparis.eu

Michel Barbeau
Carleton University, School of Computer Science, Ottawa, Canada
barbeau@scs.carleton.ca

Joaquin Garcia-Alfaro
SAMOVAR, Institut Polytechnique de Paris, Palaiseau, France
joaquin.garcia_alfaro@telecom-sudparis.eu

***Abstract*—This work introduces a single universal quantum algorithm for computing all elementary functions, with arbitrary accuracy. We leverage a technique from embedded computing and field-programmable gate arrays, called COordinate Rotation DIgital Computer (CORDIC). CORDIC is a family of iterative algorithms that, in a classical context, can approximate various trigonometric, hyperbolic, and elementary functions using only bit shifts and additions. Adapting CORDIC to the quantum context is non-trivial, as the algorithm traditionally uses several non-reversible operations. We detail a method for CORDIC that avoids such non-reversible operations. We propose an approach to calculate elementary functions reversibly with CORDIC. For n bits of precision, our method has space complexity of order n qubits, and a `CNOT` count in the order of n squared I think the depth will probably be reducible to n`polylog`(n) depending on the implementation of addition we use - huge for a Quantum CORDIC Unit (QCU) that's got data piping through. This primitive subroutine is a required step for nearly all fault algorithms including the Harrow–Hassidim–Lloyd (HHL) algorithm, quantum digital-to-analog conversion, a quantum speed-up for Monte-Carlo methods, and the quantum estimation of Shapley values.**

I. Introduction

NOTE: Extremely Rough Draft!

In the late 1950s, Volder [16] introduced the CORDIC family of algorithms to calculate some elementary functions on the limited hardware available. In the early 1970s, after the family of algorithms was expanded, Walther [17] demonstrated that the algorithms could be unified into a single algorithm. Walther showed that elementary functions such as $\sin x, \cos x, \ln, e^x, x \cdot y, x/y, \tan x, \arctan x, \sinh x, \cosh x, \sqrt{x}, etc.$ Further work by Mazenc et al. [9] demonstrated direct methods to calculate $\arcsin$, which has special importance in quantum computing due to its applications in Digital-to-Amplitude encoding [10]. This current work extends the conference article of Burge et al. [2], which calculated arcsine with a reversible, hence quantum compatible, CORDIC implementation. Using some modifications, the techniques from Burge et al. are expanded to encompass the full functionality of both Walther and Mazenc et al. Using several new optimizations, this capability expansion comes at a minimal complexity cost.

In short, this paper demonstrates a single unified and efficient quantum algorithm to calculate all elementary functions. Our method requires minimal overhead and may become a dominant approach to perform elementary function calculation for early fault tolerant quantum computing (and maybe beyond depending on a lot of specifics). Also note that, through some unintuitive tricks we leverage: (i) the double rotation structure of [9], and (ii) our MULT/DIV algorithms to normalize during each step - which leads to a whole bunch of nice stuff!

A few important points to touch on at some point:

1) For theory people: could be very nice to do a citation like *"Suppose we have the quantum CORDIC module, then we have all these functions with so-and-so resource requirements"* Resource estimation thing to look at regarding this is *Quantum algorithms: A survey of applications and end-to-end complexities* [5]. Even better, for the lazy theorists (hopefully me later this thesis), they can just be like *"By Quantum CORDIC, elementary functions can be done in $\mathcal{O}(n^2)$ CNOT gates, something like depth $\mathcal{O}(n\text{polylog}(n))$, with only $\mathcal{O}(n)$ qubits"*
2) For hardware people: you could probably make a whole company out of building only these modules. Other quantum computers could do some networking stuff to access these operations. One could, depending on the quantum technology in question, etch the algorithm directly into a circuit and then pipe queries through continuously GPU style. This gives a ballistic type of computation not far off from some QRAM suggestions - except probably much easier to implement

Section II presents related work. Section III provides an introduces the approach of classical unified CORDIC to approximate `arcsin` on minimal hardware. Section IV translates the approach to a quantum setting. Section V concludes the work.

II. Related Work

1) Basic Arithmetic review [19] - covers addition, subtraction, modular exponentiation, multiplication, and division. Can be compared somewhat to our linear CORDIC mode. Also goes over all the applications for these operations - so we'll have even more applications to list. Discusses computations basis state based arithmetic and amplitude based arithmetic. If we want to be very extra, we can compare our

complexity on exponenets, mult and div vs the ones mentioned here

2) Polynomial Based [6] - piecewise polynomial approx method. Super powerful and flexible, but requires a lot of overhead. Additionally, it is difficult to analyze the complexity, the method proposes case specific optimizations. There proposed method will work for any function that can be approximated with piecewise polynomials (which is pretty much any of them), but to do everything like our calculator they'll need additional overhead. I suspect our method will be more powerful in the short to mid term, and eventually for high precision, they'll likely be more appropriate. Note that this is not completely guaranteed though, it's possible we might be more uleful for some higher precision applications since we have linear memory scaling (while it seems like they have some sort of $\mathcal{O}(n\text{polylog}(n))$ type space complexity). We can compare our method to the arcsine, sqrt, division, tanh, exp, Gaussian
    a) The paper above, [19], cites this paper
    b) *Linear-depth quantum circuits for loading Fourier approximations of arbitrary functions* [11] and *Efficient state preparation for quantum amplitude estimation* [4] both use this - so we could talk about state prep being a use case here I suppose. There's at least two other papers on this, but it would be maybe excessive to include them all? Maybe I'll just get the published ones (non-arXiv) Here's an extra *Quantum state preparation via piecewise QSVT* [13].
    c) Of course, the next paper Wang et al. [18] cites this one.
    d) Otherwise seems like there's no citations which push this work forward on google scholar - everything else is just using this paper (based on reading the titles for the 147 citations available on google scholar)
3) Binary Search Based (maybe I should refer to it a bit differently) [18] - Designed around an iterative algorithm. Good resource on extending usable function domain. Suppose their $m = n$, seems roughly quadratically more expensive in terms of space $\mathcal{O}(n^2)$ vs our $\mathcal{O}(n)$ and like they have $\mathcal{O}(n^3)$ time complexity vs our $\mathcal{O}(n^2)$. These guys did some good work, it looks like our algorithm will be a complete improvement. They do cover the same functions as us minus the multiplication and such.
    a) Looks like we'll probably be useful for PDEs (unsurprisingly) [15]
    b) Once again, pretty much all 40 of the citations are just applying the result, not expanding it (Based on my understanding of the titles). There's a couple examples of doing related things though, but I believe they are covered in the above review [19].
4) Newton's Method Based [1] - square root, the natural logarithm, and arbitrary fractional powers. Doesn't focus on trig stuff. Makes a bunch of nice arguments on why the new quantum unified CORDIC described here will be rad. Leverages newtons method to craft division and square root function. Newton's method requires squaring operations which I think will increase space complexity and time complexity by a fair amount unless they have some tricks I'm not expecting. Here, we would compare sqrt, ln, and fractional roots (which require 3 uses of CORDIC)
    a) Of course the Polynomial based paper [6] cites this one.
    b) Improvement on the square root alg? [12], [14]
    c) Of course, the binary search based alg [18] cites this one.
    d) Looked at all 97 citations, based on titles, it looks like nothing else was pushing this one forward.
5) Poisson Paper [3] - calculates $\theta = \arcsin(\omega)$ by iteratively calculating if $\sin(\theta_k)$ is greater or equal to ω. Very similar to our method, except we're able to avoid the constant repetitions of $\sin$. Any papers that pushed this method forward:
    a) Actually cool paper I should probably do a little paragraph on: [20]. I need to read it properly, it is a totally different approach (though, I'm pretty sure we still have big advantages - e.g. we're unitary)
    b) I have checked the first 200/323 citations, nothing else so far, I'll come back to look further later.
6) Look up tables [7] - probably just bring up the critique for QRAM paper

Old related work (from when the only algorithm was arcsin) There have been a few proposed algorithms for the calculation of arcsine on quantum computers. For instance, Häner et al. [6] leverages quantum parallelism to perform piecewise polynomial approximations of a wide range of functions. It can be seen as a promising solution that scales well in precise situations, e.g., more than 32 bits. The approach is flexible, but may not be ideal in the short term, since it requires some memory access, a fair number of multiplications, and square rooting for the extreme portions of $\arcsin x$, i.e., $x \in [-1, -0.5] \cup [0.5, 1]$. There also exists the quantum function-value binary expansion method which performs a binary search to approximate $\arcsin x$, among other elementary functions [18]. The solution is broadly applicable and has some interesting parallels to the CORDIC algorithm. However, the arcsine implementation requires a squaring operation per bit of output, which is difficult to implement in the near term.

The above techniques are valuable for various applications when sufficient hardware is available. For instances where we do not have those resources available, we consider a technique from early computing. In the late 1950s, Volder [16] introduced the CORDIC algorithm to solve an important family of trigonometric equations using the underpowered hardware of the time. This was later generalized to a single algorithm that solves various elementary functions, including multiplication, division, the trigonometric functions, $\texttt{arctan}$, the hyperbolic trigonometric functions, $\ln$, $\exp$, and square root [17]. A positive trait is that many of these operations can exist with the same architecture by leveraging simple parameters [8].

CORDIC has been further modified to approximate various other functions, including our target function, `arcsin` [9].

III. Classical CORDIC for Arcsine

Algorithm 1 CORDIC Arcsine

procedure ARCSIN(t, `#iter`)▷ $t \in [-1, 1]$▷ $s(z) = 1$ if $z < 0$ else $s(z) = 0$ Here temporarily until I write up the full alg in this form
  `ang` $\leftarrow 0$, `x` $\leftarrow 1$, `y` $\leftarrow 0$, `t` $\leftarrow t$, `d` $\leftarrow 0$
  for $i = 1, i <$ `#iter`, `++`i **do**
    $\mathtt{d}_i \leftarrow [s(\mathtt{x}) \wedge s(\mathtt{t} - \mathtt{y})] \oplus s(\mathtt{x}) \oplus [s(\mathtt{x}) \wedge s(\mathtt{y})] \oplus s(\mathtt{t} - \mathtt{y})$
    if $\mathtt{d}_i$ **then** $\mathtt{x}, \mathtt{y} \leftarrow \mathtt{y}, \mathtt{x}$
    for `_ in range(2)` **do**
      $\mathtt{x}, \mathtt{y} \leftarrow \mathtt{x} - 2^{-i}\mathtt{y}, \mathtt{y} + 2^{-i}\mathtt{x}$
    if $\mathtt{d}_i$ **then** $\mathtt{x}, \mathtt{y} \leftarrow \mathtt{y}, \mathtt{x}$
    $\mathtt{t} \leftarrow (1 + 2^{-2i})\mathtt{t}$
    $\mathtt{ang} \leftarrow \mathtt{ang} + (-1)^{\mathtt{d}_i} 2 \arctan 2^{-i}$
  return `ang`

A. Non-reversible Unified CORDIC

Boiled down to it's most essential parts, CORDIC can be described abstractly as the following routine:

1) Begin with initial vector (x_0, y_0) with associated angle z_0. We want to approximate some goal vector (x, y) z where x, y, or z is the desired result.
2) Based on current state (x_k, y_k), z_k, and the goal vector, decide which direction to evolve the vector
3) Evolve the vector to construct (x_{k+1}, y_{k+1}) and z_{k+1}.
4) Repeat Steps 2 and 3. As $k \to \infty$, we have, $x_k, y_k, z_k \to x, y, z$.

This section gives a summary of Walther's [17] unified CORDIC algorithm (partly using description of Walther's algorithm from [9]). Walther's algorithm united several algorithms in the CORDIC family. The algorithm has two parameters. Parameter m represents three modes: $(m = 1)$ is circular mode; $(m = 0)$ is linear mode; and, $(m = -1)$ is hyperbolic mode. The second parameter $\mathcal{D} : \mathbb{R}^3 \to \{0, 1\}$ has two modes: $\mathcal{D}(x, y, z) = \mathtt{isneg}(y)$ is rotation mode; and $\mathcal{D}(x, y, z) = \mathtt{isneg}(z)$ is vectoring mode. The m parameter controls the coordinate system, while the $\mathcal{D}$ parameter decides what direction to rotate towards. We define a sketch of the Walther algorithm as follows,

$\mathtt{CORDIC}_{m,\mathcal{D}}(x_0, y_0, z_0)$

1) Begin with inputs x_0, y_0, z_0, and parameters m and $\mathcal{D}$.
2) Calculate rotation direction $d_k = \mathcal{D}(x_k, y_k, z_k)$.
3) Perform rotation,
$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \frac{1}{\sqrt{1 + m\gamma_k^2}} \begin{pmatrix} 1 & -m d_k \gamma_k \\ d_k \gamma_k & 1 \end{pmatrix} \begin{pmatrix} x_k \\ y_k \end{pmatrix}$$
Where γ_k is a decreasing positive sequence.
4) Update $z_{k+1} = z_k + d_k \alpha_k$, where $\alpha_k = \tan^{-1} \gamma_k$ in circular mode, $\alpha_k = \gamma_k$ in linear mode, and $\alpha_k = \tanh^{-1} \gamma_k$ in hyperbolic mode. Note that α_k is precomputed.
5) Repeat Steps 2 to 4 for $k \in \{1, \ldots, n\}$, where $n = \mathcal{O}(\log \epsilon^{-1})$, for ϵ accuracy.

For asymptotic outputs of $\mathtt{CORDIC}_{m,\mathcal{D}}(x, y, z)$, see Figure 1.

By choosing the correct $m, \mathcal{D}, x, y, z$ combinations, one can trivially perform functions $xy, x/y, \sin x, \cos x, \tan^{-1} x, \sinh x, \cosh x, \tanh^{-1} x$. We can easily perform $e^x = \sinh x + \cosh x$, and $\ln x = \tanh^{-1} \frac{x+1}{x-1}$. Square root is also available at the cost of one application of the CORDIC module with $\sqrt{x} = \sqrt{(x + 1/4)^2 - (x - 1/4)^2}$. One can also construct $\tan x = \frac{\sin x}{\cos x}$ and $\tanh x = \frac{\sinh x}{\cosh x}$. The above identities are all described in Walther [17]. Additionally, with three applications of CORDIC, we can apply fractional powers x^y with the identity $x^y = e^{y \ln x}$. One can also derive the inverse trig and hyperbolic functions from the available operations. However, in the next section, we will describe a modification which allows for native calculation of $\sin^{-1}$ and $\sinh^{-1}$ (we won't have native support for $\cosh$ unless we want to add a bit of extra complexity, it'll take like 3 CORDICs or so with the archetecture I'm thinking). I'm pretty confident that this means we can do all the elementary functions.

NOTE: in Walther's original algorithm, the normalization is saved for the end and performed with CORDIC in linear mode. We found a way to normalize on each iteration more efficiently, leveraging `DIV`, `MULT` and some sneaky double rotation trickery.

IV. Quantum CORDIC for Arcsine

We first describe a reversible method to perform walther's [17] algorithm:

1) For Step 1, we initialize resisters `x`, `y`, `z` with x_0, y_0, z_0. Each register stores an n-bit fixed point two's compliment binary number in range $[-2, 2)$, e.g.,
$$x_0 = x_0^{(n-1)} x_0^{(n-2)} . x_0^{(n-3)} \cdots x_0^{(1)} x_0^{(0)}.$$
We additionally initialize an $(n-1)$-bit register `d` to store rotation directions, and an n-bit auxiliary register `MULT`. Finally, we have parameters $m, \mathcal{D}$, stored in registers `m`, `D` of size two and one respectively.
2) One can easily perform Step 2 by leveraging the two's compliment representation of registers `y`, `z`, where `isneg(y)`$=y^{(n-1)}$ and `isneg(z)`$=z^{(n-1)}$. So we simply perform,
$$\begin{aligned} &|\mathcal{D}\rangle_{\mathtt{D}} |x_k\rangle_{\mathtt{x}} |y_k\rangle_{\mathtt{y}} |z_k\rangle_{\mathtt{z}} |0\rangle_{\mathtt{d}(k)} \\ &\to |\mathcal{D}\rangle_{\mathtt{D}} |x_k\rangle_{\mathtt{x}} |y_k\rangle_{\mathtt{y}} |z_k\rangle_{\mathtt{z}} |(z_k^{(n-1)} \wedge \mathcal{D}) \oplus (y_k^{(n-1)} \wedge \neg\mathcal{D})\rangle_{\mathtt{d}(k)}. \end{aligned}$$
Note that this can be performed with two Toffoli gates and two Not gates.
3) The rotation requires the most sneaky tricks. Suppose we have access to Algorithm 2,
$$\mathtt{DIV}(\ell, c) |r\rangle = |(1 + (-1)^c 2^{-\ell})^{-1} r\rangle.$$
We denote it's inverse $\mathtt{MULT}(\ell, c) = \mathtt{DIV}(\ell, c)^\dagger$. We break Step 3 into two in Figure 2.
These two operations correspond directly to the normalized rotation required in our description of Walther's algorithm [17] Step 3.

	Rotation ($\mathcal{D} = \texttt{isneg(y)}$)	Vectoring $\mathcal{D} = \texttt{isneg(z)}$
Circular ($m = 1$)	$(\sqrt{x^2 + y^2}, 0, z + \tan^{-1}(y/x))$	$(x \cos z - y \sin z, x \cos z + y \sin z, 0)$
Linear ($m = 0$)	$(x, 0, z + x/y)$	$(x, y + x \cdot z, 0)$
Hyperbolic ($m = -1$)	$(\sqrt{x^2 - y^2}, 0, z + \tanh^{-1}(y/x))$	$(x \cosh z + y \sinh z, y \cosh z + x \sinh z, 0)$

Fig. 1: Walther's asymptotic unified CORDIC outputs for each parameter config and inputs x, y, z [17].

a) Perform matrix,

$$\begin{pmatrix} x'_k \\ y_k \end{pmatrix} = \sqrt{1 + (-1)^{(m==-1)}\gamma_k^2} \begin{pmatrix} 1 & (-1)^{\texttt{not } d^{(k)} \oplus (m==-1)} 2^{-k} \\ (-1)^{\texttt{not } d^{(k)}} 2^{-k} & 1 \end{pmatrix} \begin{pmatrix} x_k \\ y_k \end{pmatrix}$$

This step is performed with (i): a shifted addition from the y register to the x register, leveraging bitwise not trick to subtract if $\texttt{not } d^{(k)} \oplus (m == -1)$, is TRUE. Then, (ii): MULT is applied to the y register to compensate for the following step. Then (iii): a shifted addition from the x register to the y register, leveraging bitwise not trick to subtract if $\texttt{not } d^{(k)}$, is TRUE.

b) Perform matrix,

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = \frac{1}{1 + (-1)^{(m==-1)}\gamma_k^2} \begin{pmatrix} 1 & (-1)^{\texttt{not } d^{(k)} \oplus (m!=1)} 2^{-k} \\ (-1)^{\texttt{not } d^{(k)}} 2^{-k} & 1 \end{pmatrix} \begin{pmatrix} x'_k \\ y'_k \end{pmatrix}$$

This step is performed with (i): a shifted addition from the y register to the x register, leveraging bitwise not trick to subtract if $\texttt{not } d^{(k)} \oplus (m! = 1)$, is TRUE. Note the logic here is slightly different than with the m, this is so when $m == 0$ (linear mode), the operations on x cancel out! Then, (ii): DIV is applied to the x register to compensate for the following step, and to normalize the two pseudo-rotations. The DIV is a super-sneaky trick, since it undoes the stretch associated with the previous pseudo-rotation, the idea is we a normalization after is equivalent to cancelling out the MULT in the y reg and applying DIV to the x register. Then (iii): a shifted addition from the x register to the y register, leveraging bitwise not trick to subtract if $\texttt{not } d^{(k)}$, is TRUE.

Fig. 2: Reversible method to perform Walther's Step 3.

4) With 3 controlled additions (6 additions plus some bitwise nots), we can perform the z register update from Step 4. With an extra register we can replace the six additions with one combined with some data-loading CNOTS.
5) Repeats steps for $k = 1, 2, 3, 4, 4, 5, \cdots, 11, 12, 13, 13, 14, \cdots$. Note that steps $\{4, 13, 40, 121, \cdots, k, 3k + 1, \cdots\}$ are repeated. Note that I'll need a different value for current step and shift value because this notation is not quite correct

The above is a slightly stripped down version of the algorithm in this github repo

V. Conclusion

Once again, this is a very rough draft, we'll have a proper preprint in a few weeks

We have introduced a Quantum algorithm for calculating elementary functions with low space and time complexity. Of course, these have wide applications, arcsine alone has applications in HHL (Harrow–Hassidim–Lloyd), approximating Shapley values, and quantum Monte Carlo methods. It seems likely that our algorithm will find use for low to medium precision operations on early fault tolerant quantum hardware and perhaps beyond.

References

[1] Mihir K Bhaskar, Stuart Hadfield, Anargyros Papageorgiou, and Iasonas Petras. Quantum algorithms and circuits for scientific computing. *arXiv preprint arXiv:1511.08253*, 2015.

[2] Iain Burge, Michel Barbeau, and Joaquin Garcia-Alfaro. Quantum cordic – arcsine on a budget. In *2026 IEEE International Conference on Quantum Computing and Engineering (QCE)*, volume 1, pages 1–8. IEEE, 2026.

[3] Yudong Cao, Anargyros Papageorgiou, Iasonas Petras, Joseph Traub, and Sabre Kais. Quantum algorithm and circuit design solving the Poisson equation. *New Journal of Physics*, 15(1):013021, January 2013.

[4] Almudena Carrera Vazquez and Stefan Woerner. Efficient state preparation for quantum amplitude estimation. *Physical Review Applied*, 15(3):034027, 2021.

[5] Alexander M Dalzell, Sam McArdle, Mario Berta, Przemyslaw Bienias, Chi-Fang Chen, András Gilyén, Connor T Hann, Michael J Kastoryano, Emil T Khabiboulline, and Aleksander Kubica. Quantum algorithms: A survey of applications and end-to-end complexities. Technical report, 2025.

[6] Thomas Häner, Martin Roetteler, and Krysta M Svore. Optimizing quantum circuits for arithmetic. *arXiv preprint arXiv:1805.12445*, 2018.

[7] Samuel Jaques and Arthur G Rattew. Qram: A survey and critique. *Quantum*, 9:1922, 2025.

[8] Boppana Lakshmi and Anindya Sundar Dhar. Cordic architectures: A survey. *VLSI design*, 2010(1):794891, 2010.

[9] Christophe Mazenc, Xavier Merrheim, and J-M Muller. Computing functions cos/sup-1/and sin/sup-1/using cordic. *IEEE Transactions on Computers*, 42(1):118–122, 1993.

[10] Kosuke Mitarai, Masahiro Kitagawa, and Keisuke Fujii. Quantum analog-digital conversion. *Physical Review A*, 99(1):012301, 2019.

[11] Mudassir Moosa, Thomas W Watts, Yiyou Chen, Abhijat Sarma, and Peter L McMahon. Linear-depth quantum circuits for loading fourier approximations of arbitrary functions. *Quantum Science and Technology*, 9(1):015002, 2024.

[12] Edgard Muñoz-Coreas and Himanshu Thapliyal. T-count and qubit optimized quantum circuit design of the non-restoring square root algorithm. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 14(3):1–15, 2018.

[13] Oliver O'Brien and Christoph Sünderhauf. Quantum state preparation via piecewise qsvt. *Quantum*, 9:1786, 2025.

[14] Afrin Sultana and Edgard Muñoz-Coreas. Qubit and t-count optimized quantum circuit design for fixed precision square root. In *2024 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, pages 650–655. IEEE, 2024.

[15] Giorgio Tosti Balducci, Boyang Chen, Matthias Möller, Marc Gerritsma, and Roeland De Breuker. Review and perspectives in quantum

computing for partial differential equations in structural mechanics. *Frontiers in Mechanical Engineering*, 8:914241, 2022.

[16] Jack E Volder. The cordic trigonometric computing technique. *IRE Transactions on electronic computers*, (3):330–334, 1959.

[17] John Stephen Walther. A unified algorithm for elementary functions. In *Proceedings of the May 18-20, 1971, spring joint computer conference*, pages 379–385, 1971.

[18] Shengbin Wang, Zhimin Wang, Wendong Li, Lixin Fan, Guolong Cui, Zhiqiang Wei, and Yongjian Gu. Quantum circuits design for evaluating transcendental functions based on a function-value binary expansion method. *Quantum Information Processing*, 19:1–31, 2020.

[19] Siyi Wang, Xiufan Li, Wei Jie Bryan Lee, Suman Deb, Eugene Lim, and Anupam Chattopadhyay. A comprehensive study of quantum arithmetic circuits. *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, 383(2288), 2025.

[20] Nathan Wiebe and Martin Roetteler. Quantum arithmetic and numerical analysis using repeat-until-success circuits. *arXiv preprint arXiv:1406.2040*, 2014.

APPENDIX

A. MULT FUNCTION FOR C=0

I'll update to work with c=1, this proof is just lifted from our QCE2026 pub so far

To understand the `Mult` algorithm, it is more instructive to considered its inverse, `Div` (Algorithm 2). To simplify the following proofs, we assume infinite bits of precision in our number representations.

Lemma A.1. *Suppose i is even, at the beginning of the ith iteration of Algorithm 2's for loop, the state is,*

$$in = z \sum_{k=0}^{F[i+1]-1} r^k, \quad aux = z \sum_{k=0}^{F[i]-1} r^k.$$

where z is the initial value of `in`*, and $r = -2^{-m}$.*

Proof: We proceed by induction. Base case: for $i = 0$, at the beginning of the loop $\texttt{in} = \texttt{aux} = zr^0$. Thus, the base case holds.

Inductive step: suppose that for an even $i < \lceil \sqrt{5}\varphi^{-m}n \rceil$, the registers hold the values,

$$\texttt{in} = z \sum_{k=0}^{F[i+1]-1} r^k, \quad \texttt{aux} = z \sum_{k=0}^{F[i]-1} r^k.$$

i is even so the logical statement in Line 6 is true. Hence, `aux` is updated to,

$$\texttt{aux} \leftarrow z \sum_{k=0}^{F[i]-1} r^k + (-1)^{\texttt{F}[i]} \cdot 2^{m\texttt{F}[i]} z \sum_{k=0}^{F[i+1]-1} r^k.$$

By definition $(-1)^{\texttt{F[i]}} \cdot 2^{m\texttt{F}[i]} = r^{\texttt{F}[i]}$, thus, the new value for `aux` is $z\sum_{k=0}^{F[i]-1} r^k + z\sum_{k=F[i]}^{F[i+1]+F[i]-1} r^k$. By definition of the Fibonacci sequence, $F[i+2] = F[i+1] + F[i]$, combining the summations yields,

$$\texttt{aux} = z \sum_{k=0}^{F[i+2]-1} r^k.$$

On the next loop, the index register holding i incremented by 1 and is now odd, so the if statement on Line 6 gets false. Hence, the next operation updates `in`,

$$\texttt{in} \leftarrow z \sum_{k=0}^{F[i+1]-1} r^k + (-1)^{\texttt{F}[i+1]} \cdot 2^{m\texttt{F}[i+1]} z \sum_{k=0}^{F[i+2]-1} r^k$$

Similarly to the previous iteration, we find `in` is equal to $z\sum_{k=0}^{F[i+1]-1} r^k + z\sum_{k=F[i+1]}^{F[i+2]+F[i+1]-1} r^k$. Bringing the summations together shows,

$$\texttt{in} = z \sum_{k=0}^{F[i+3]-1} r^k.$$

The index register is incremented once again, such that it holds $i + 2$. By induction, the result holds. ∎

Remark A.1. *Recall, for $r \in (0, 1)$, and an integer J, the geometric sum formula gives,*

$$\sum_{k=0}^{J-1} r^k = \frac{1 - r^J}{1 - r} = \frac{1 - 2^{-Jm}}{1 + 2^{-m}}.$$

Algorithm 2 Div(in, aux, ℓ, c): $\texttt{in} \leftarrow (1 + (-1)^c 2^{-\ell})^{-1}\texttt{in}$

1: **procedure** DIV(in, aux, ℓ) ▷ in is a register of size n, aux is a auxiliary register of size n with near 0 value, ℓ is the right shift, and c decides $+/-$
2: F $\leftarrow [1, 1, 2, 3, 5, 8, 13, \dots]$ ▷ Fibbonacci Sequence
3: #iter $\leftarrow 2\left\lceil \log_\varphi(\sqrt{5}n/\ell)/2 \right\rceil$ ▷ $\varphi = (1 + \sqrt{5})/2$
4: aux $\leftarrow$ aux + in
5: **for** $i = 0$, $i <$ #iter, $++i$ **do**
6: **if** i even **then**
7: aux $\leftarrow$ aux $+ (-1)^{(\neg c)\texttt{F}[i]+c}(\texttt{in} \gg (\ell \cdot \texttt{F}[i]))$
8: **else**
9: in $\leftarrow$ in $+ (-1)^{(\neg c)\texttt{F}[i]+c}(\texttt{aux} \gg (\ell \cdot \texttt{F}[i]))$
10: aux $\leftarrow$ aux − in

Theorem A.2. *Assuming J iterations, the final state is,*

$$in \approx \frac{z}{1 - 2^{-m}}, \quad aux \approx 0,$$

with an accuracy of $\mathcal{O}\left(m\varphi^J\right)$ bits.

Proof: First note the state prior to Line 10 is,

$$\texttt{in} = z \sum_{k=0}^{\texttt{F}[J+1]-1} r^k, \quad \texttt{aux} = z \sum_{k=0}^{\texttt{F}[J]-1} r^k.$$

For simplicity, we assume J is even and greater than or equal to two. Then, Line 10 results in state,

$$\texttt{aux} \leftarrow z \sum_{k=0}^{\texttt{F}[J]-1} r^k - z \sum_{k=0}^{\texttt{F}[J+1]-1} r^k.$$

As a result, `aux` is equal to $-z\sum_{k=\texttt{F}[J]}^{\texttt{F}[J+1]-\texttt{F}[J]-1} r^k$. This can be simplified to, $-zr^{\texttt{F}[J]}\sum_{k=0}^{\texttt{F}[J-1]-1} r^k$. Then, the geometric sum formula gives,

$$\texttt{aux} = -zr^{\texttt{F}[J]}\frac{1 - r^{\texttt{F}[J-1]}}{1 - r}.$$

Note that $\varphi^{k-1} < \mathtt{F}[k]$ for positive k. Thus,

$$\mathtt{aux} < -z2^{-\varphi^{J-1}m}\frac{1-2^{-\varphi^{J-2}m}}{1-2^{-m}}.$$

Note that z is always less than 2 as argued at the beginning of Section IV. Since $m \geq 1$, $J \geq 2$, we have $|aux| \leq 2^{-\varphi^{J-1}m+2}$. Thus, the error introduced to `aux` in terms of bit precision is exponentially small.

On the other hand, the `in` register state is,

$$\mathtt{in} = z\frac{1-2^{-mF[J]}}{1+2^{-m}}.$$

Thus, we have an absolute error of less than, $2 \cdot 2^{-mF[J]} \leq 2^{-m\varphi^{J-1}}$. Therefore, the absolute error is exponentially small with respect to number of iterations. ■